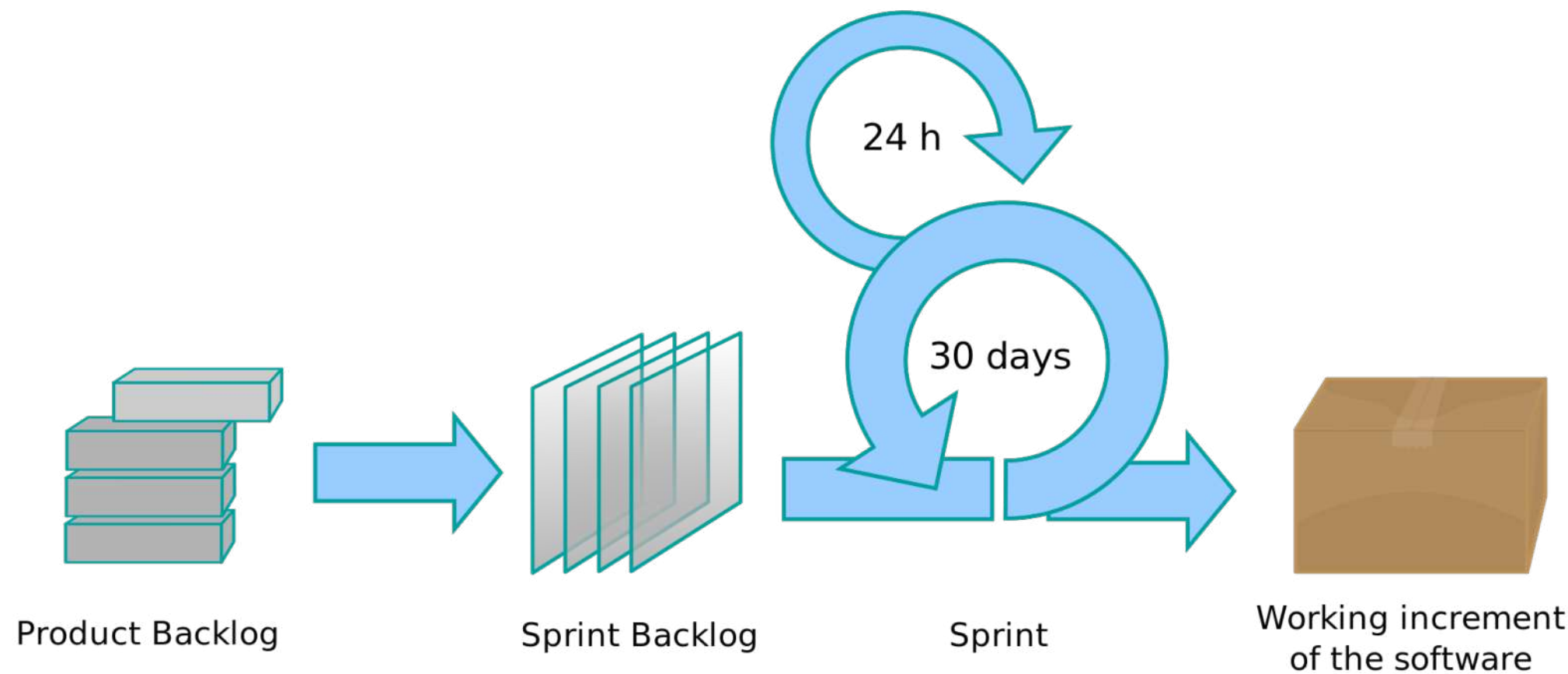
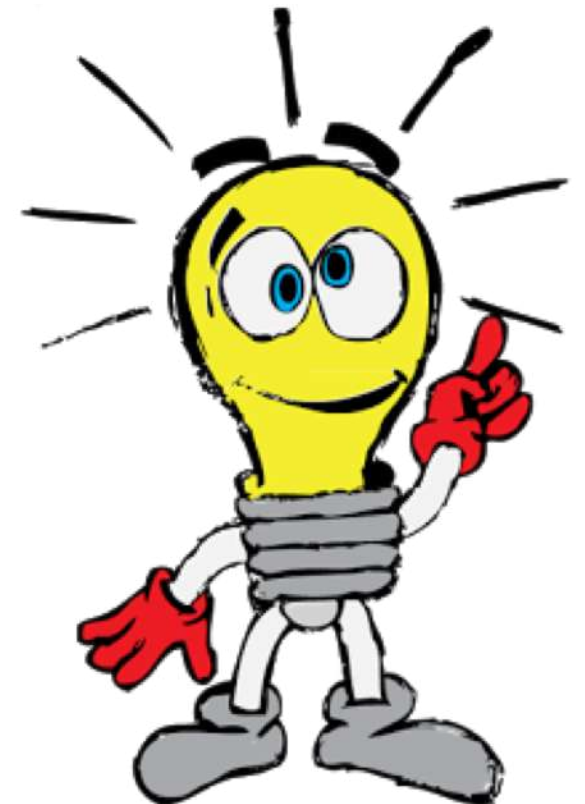
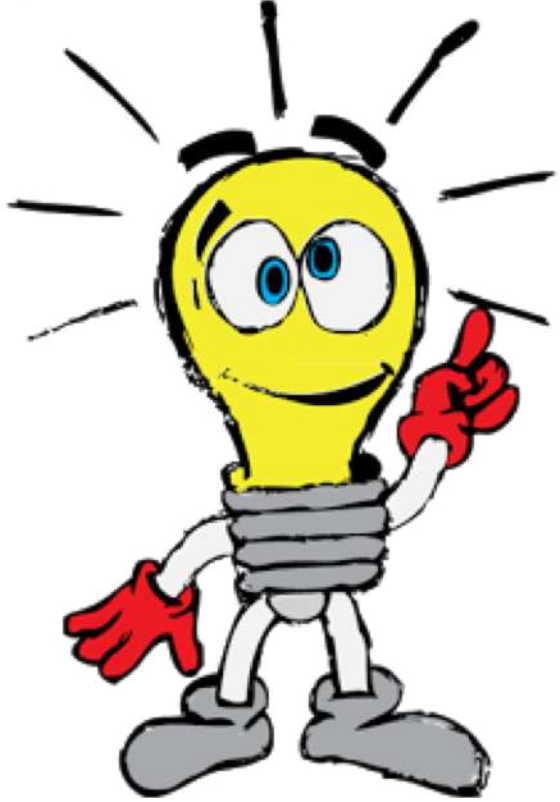


การพัฒนาซอฟต์แวร์แบบไฮสปีดด้วยวิธีสกรัม



Agile ไม่ได้แก้ปัญหาเรื่อง Requirement change, ระบบช้า, Bug
หรืออะไรที่เคยเจอใน SDLC ก็ยังจะเจอใน Agile





Programmer ԲՉՏ Get Requirement ԻՅՆ!

agile | 'adʒaɪl |

adjective

- 1 able to move quickly and easily: *Ruth was as agile as a monkey.*
 - able to think and understand quickly: *his vague manner concealed an agile mind.*
- 2 relating to or denoting a method of project management, used especially for software development, that is characterized by the division of tasks into short phases of work and frequent reassessment and adaptation of plans: *agile methods replace high-level design with frequent redesign.* Contrasted with **WATERFALL (ADJECTIVE)**.

(adj) กระฉับกระเฉง

คำเหมือน: quick, nimble, spry

History of Agile

In the late 1990's several methodologies began to get increasing public attention. Each had a different combination of old and new ideas, but all emphasized:

- Close collaboration between the programmer team and business experts
- Face-to-face communication (as more efficient than written documentation)
- Frequent delivery of new deployable business value
- Tight, self-organizing teams
- Ways to craft the code and the team so the inevitable requirements churn was not a crisis

Who?	17 independent thinkers, competitors and anarchists from XP, Scrum, DSDM, ASD, et al
What?	Met to try to find common ground on software development
When?	February 11-13, 2001
Where?	The Lodge at Snowbird, Utah
Why?	They felt the need for an alternative to "documentation-driven, heavyweight software development processes"
How?	They skied, ate, drank, and came to terms, naming themselves "The Agile Alliance"
Output?	The Agile Software Development Manifesto



Kent Beck
Martin Fowler
Jon Kern
Robert C. Martin

Mike Beedle
James Grenning
Steve Mellor

Arie van Bennekum
Jim Highsmith
Jeff Sutherland

Alistair Cockburn
Andrew Hunt
Ken Schwaber

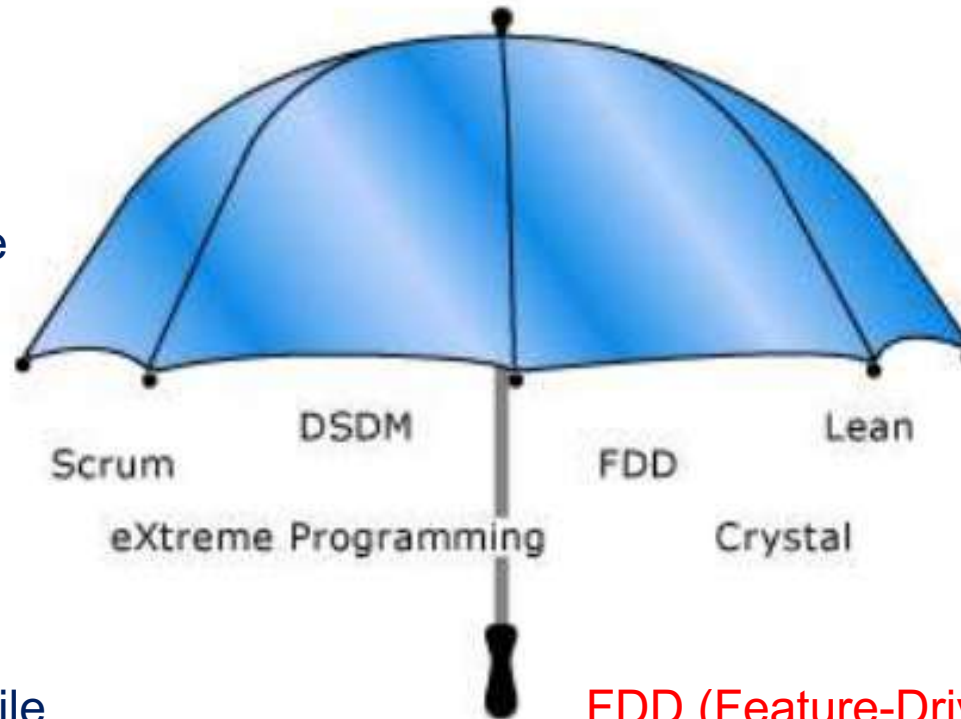
Ward Cunningham
Ron Jeffries
Brian Marick
Dave Thomas

Dynamic System Development Method (DSDM). an iterative and incremental approach that embraces principles of Agile development, including continuous user/customer involvement.

Scrum. A term borrowed from the sport of rugby. 1. A lightweight agile framework for managing complex product and service development. 2. An iterative and incremental approach to developing products and managing work.

eXtreme Programming (XP). An agile development approach that is complementary to Scrum. Extreme Programming specifies important technical practices that development teams use to manage that flow of task-level work during sprint execution.

And Agile umbrella

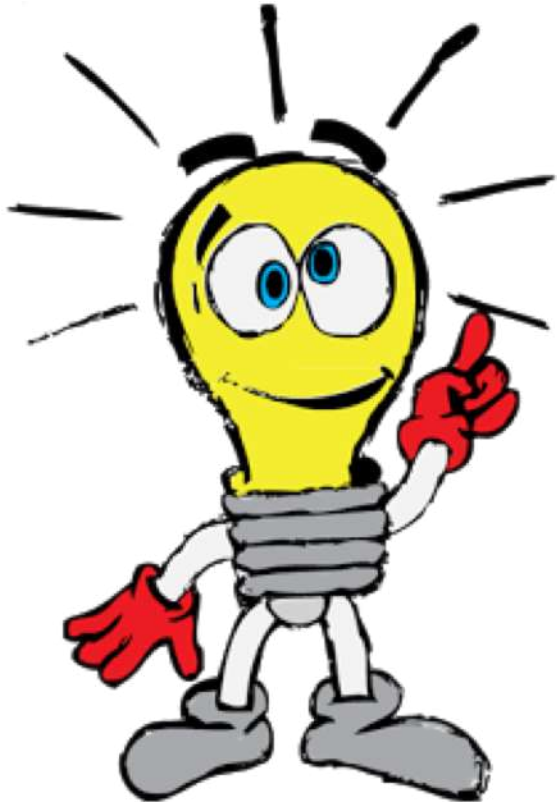


Crystal. focuses on people, interaction, community, skills, talents, and communications as first-order effects on performance. Process remains important, but secondary.

Lean. Lean comes from Lean Manufacturing and is a set of principles for achieving quality, speed & customer alignment.

- | | |
|---------------------|-----------------------|
| 1. Eliminate Waste | 5. Deliver Fast |
| 2. Build Quality In | 6. Respect People |
| 3. Create Knowledge | 7. Optimize the Whole |
| 4. Defer Commitment | |

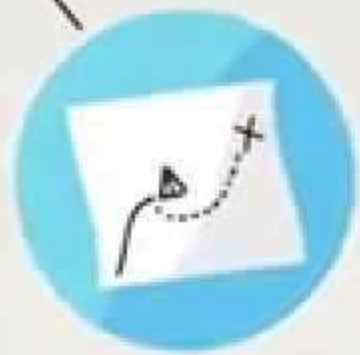
FDD (Feature-Driven Development). 1. A slice of business functionality that is meaningful to a customer or user. 2. Used by some to mean a medium-size user story that can and will be **divided into a collection of smaller** user stories that together will be implemented to deliver that value of a feature.



- คนในทีมควรมีความรู้หลากหลาย ทั้ง Business และ Technology
- อย่าพยายามทำเป็น Resource Pool*
- cross-functional team

Cross-functional team : A team composed of members with all the functional skills (such as UI designers, developers, testers) and specialities necessary to complete work that requires more than a single discipline.

AGILE



Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

คำแถลงอุดมการณ์แห่งอไจล์

เราค้นพบวิธีที่ดีกว่าในการพัฒนาซอฟต์แวร์
จากการลงมือทำจริงและช่วยเหลือผู้อื่น
นั่นคือ เราให้ความสำคัญกับ:

คนและการมีปฏิสัมพันธ์กัน มากกว่าการทำตามขั้นตอนและเครื่องมือ
ซอฟต์แวร์ที่นำไปใช้งานได้จริง มากกว่าเอกสารที่ครบถ้วนสมบูรณ์
ร่วมมือทำงานกับลูกค้า มากกว่าการต่อรองให้เป็นไปตามสัญญา
การตอบรับกับการเปลี่ยนแปลง มากกว่าการทำตามแผนที่วางไว้

ทั้งนี้ แม้เราจะเห็นความสำคัญในสิ่งที่กล่าวไว้ทางด้านขวา
แต่เราให้ความสำคัญกับสิ่งที่กล่าวไว้ทางด้านซ้ายมากกว่า

คนและการมีปฏิสัมพันธ์กัน

มากกว่าการทำตามขั้นตอนและเครื่องมือ

Individuals and interactions over processes and tools

❓ เพราะว่า

ทักษะและความสามารถของตัวบุคคลนั้นสำคัญมากในการพัฒนาซอฟต์แวร์ ดังนั้นไม่มีกระบวนการ ขั้นตอน และเครื่องมือใดๆ ที่จะสามารถนำมาใช้เพื่อทดแทนกันได้

ทีมพัฒนาซอฟต์แวร์จะต้องประกอบไปด้วยคนที่มีทักษะที่หลากหลายมาทำงานร่วมกันจึงจะประสบความสำเร็จ

💡 ดังนั้น

ต้องเสริมทักษะด้วยการจัดหาผู้เชี่ยวชาญและโค้ชมาให้องค์ความรู้ ฝึกฝน และปฏิบัติด้วยการลงมือทำจริง ๆ เพื่อเพิ่มพูนทักษะ ความสามารถและประสบการณ์ให้กับสมาชิกของทีม

เมื่อสมาชิกในทีมมีทักษะและความสามารถเพิ่มขึ้น ให้ทีมเป็นผู้คิดและตัดสินใจเลือกใช้กระบวนการหรือสร้างกระบวนการขั้นตอนการทำงาน พร้อมทั้งเครื่องมือที่เหมาะสม

ซอฟต์แวร์ที่ทำงานได้จริง

มากกว่าเอกสารที่ครบถ้วนสมบูรณ์

Working software over comprehensive documentation

❓ เพราะว่า

เมื่ออ่านเอกสารผู้อ่านต้องตีความและจินตนาการว่าซอฟต์แวร์ตามที่บรรยายไว้ในเอกสารเป็นเช่นไร

ต่างคน ต่างความคิด ต่างการตีความ ต่างการจินตนาการ เมื่ออ่านเอกสารฉบับเดียวกัน

💡 ดังนั้น

ลงมือพัฒนาซอฟต์แวร์ที่ละส่วนเล็กๆ โดยใช้เอกสารประกอบเท่าที่จำเป็นและเพียงพอในการช่วยการออกแบบและพัฒนาเพื่อตรวจสอบว่าถูกต้องและตรงตามความต้องการของลูกค้าหรือไม่

สร้างเอกสารที่ใช้ประกอบการพัฒนาซอฟต์แวร์อย่างตั้งใจเพื่อส่งมอบแต่ผู้ที่มารับผิดชอบต่อและพัฒนาซอฟต์แวร์ต่อไป

แหล่งที่มา: เอกสารการสอน Passionate Product Owner ของ Jeff Patton

ร่วมมือทำงานกับลูกค้า

มากกว่าการต่อรองให้เป็นไปตามสัญญา

Customer collaboration over contract negotiation

❓ เพราะว่า

ลูกค้า ผู้ที่ต้องการใช้งานซอฟต์แวร์นั้นๆ จริงๆ แล้วก็ไม่สามารถที่จะคิดและบอกความต้องการได้ครบถ้วนและสมบูรณ์

การบังคับให้ลูกค้าเขียนอธิบายความต้องการทั้งหมดอย่างละเอียดลงในเอกสารแล้วก็นำเอกสารนั้นมาพูดคุยต่อรองเพื่อให้ได้ผลประโยชน์ทั้งสองฝ่ายเป็นการเสียเวลาไปอย่างเปล่าประโยชน์

💡 ดังนั้น

ใช้การทำงานร่วมกันอย่างใกล้ชิดและการพูดคุยปรึกษาหารือเพื่อสื่อสารความต้องการ

ใช้การตรวจสอบและทดสอบซอฟต์แวร์ที่ทำงานได้จริงเพื่อยืนยันว่าถูกต้องและครบถ้วนตามความต้องการที่ตกลงกัน

แหล่งที่มา: เอกสารการสอน Passionate Product Owner ของ Jeff Patton

การตอบรับการเปลี่ยนแปลง

มากกว่าการทำตามแผนที่วางไว้

Responding to change over following a plan

❓ เพราะว่า

การวางแผนล่วงหน้าไกลๆ คือการคาดการณ์ ทำนาย และทายทัก สิ่งที่เราคาดว่าจะเกิดขึ้น ซึ่งต้องยอมรับว่ามนุษย์แบบเราๆ คาดการณ์ ทำนายและทายทัก อะไร ไกลๆ ได้แย่มาก

เมื่อไรที่มีการลงโทษหรือประจานใครที่คาดการณ์ ทำนายและทายทัก แผนงานหรือแผนการผิด เขาก็จะหยุดที่จะตรวจสอบแต่จะเริ่มที่จะซ่อนความผิดพลาด และข้อมูลที่สำคัญๆ

💡 ดังนั้น

ปรับการวางแผนเป็นสองระยะ วางแผนระยะยาวเพื่อให้เห็นภาพรวมทั้งหมดและมีจุดตรวจสอบเป็นระยะและลงรายละเอียดในการวางแผนระยะสั้น

ตรวจสอบผลลัพธ์ที่เกิดขึ้นจากการดำเนินการตามแผนระยะสั้นเพื่อจะได้เรียนรู้และนำมาใช้ในการวางแผนหรือปรับปรุงแผนระยะสั้นที่จะเกิดขึ้นต่อไป

แหล่งที่มา: เอกสารการสอน Passionate Product Owner ของ Jeff Patton

12 Principles of Agile Software Development

01 Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.



02 Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.



03 Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.



Deliver working software frequently

Release It

Test It

Build It

Continuously

04 Business people and developers must work together daily throughout the project.



05 Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.





06

Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

07 Working software is the primary measure of progress.



08

The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.



09

Continuous attention to technical excellence and good design enhances agility.



SPECIFICATION BY EXAMPLE



CONTINUOUS
INTEGRATION



CONTINUOUS DELIVERY



TEST AUTOMATION



TECHNICAL
EXCELLENCE



ARCHITECTURE
& DESIGN



ACCEPTANCE
TESTING



THINKING ABOUT TESTING



TEST-DRIVEN DEVELOPMENT

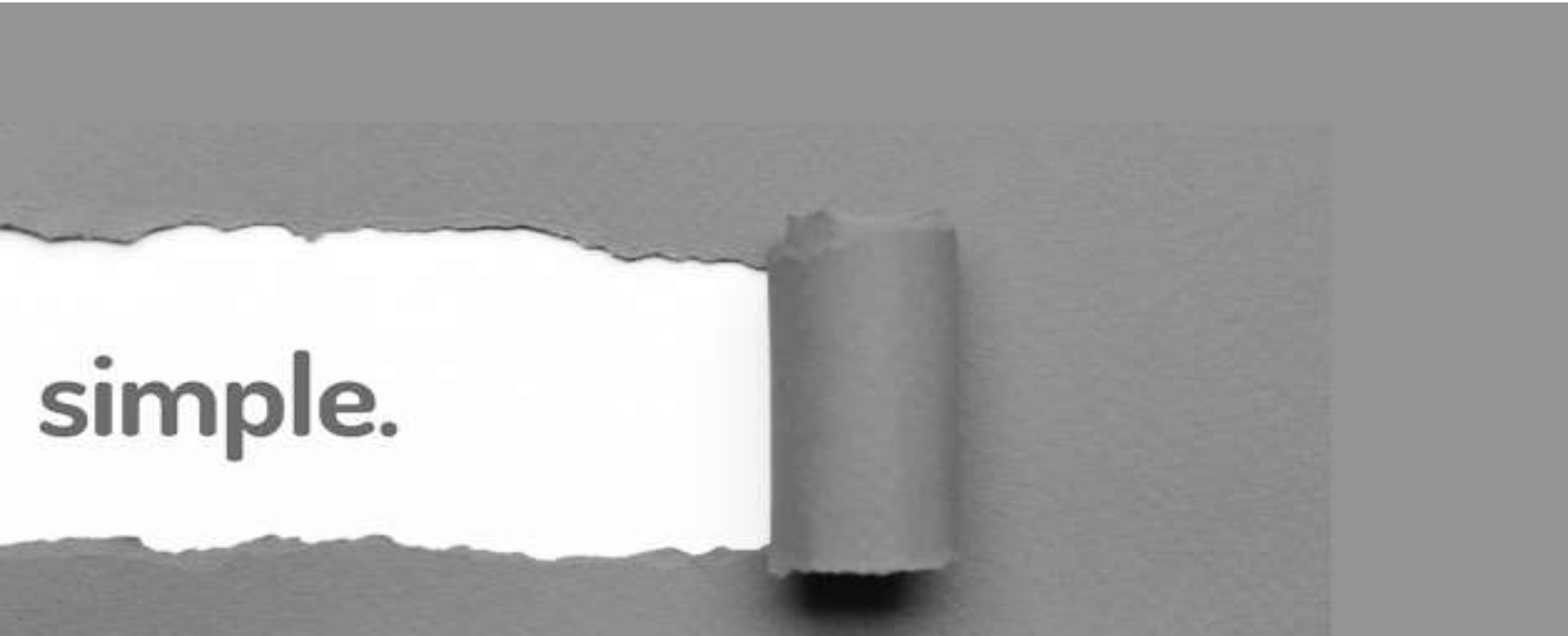
CODE

CLEAN CODE



UNIT TESTING

10 Simplicity—the art of maximizing the amount of work not done—is essential.

A grayscale photograph of a piece of paper with a torn, deckled edge. The paper is light gray and is set against a darker gray background. On the left side of the paper, the word "simple." is written in a bold, lowercase, sans-serif font. The paper appears to be slightly curled or folded, with a shadow visible on the right side.

simple.

11 The best architectures, requirements,
and designs emerge from self-organizing
teams.

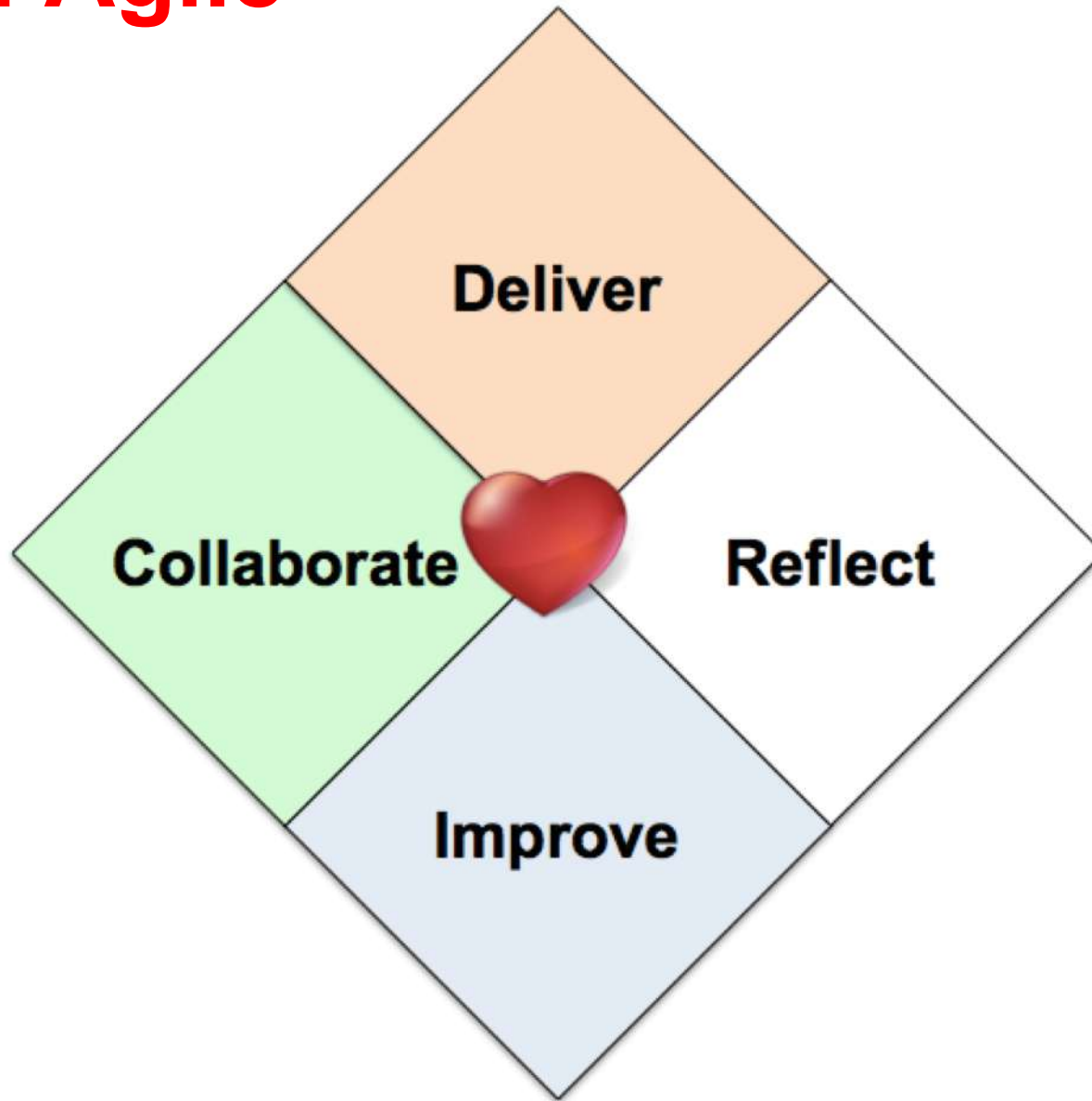
VDO clip
The Power of Teamwork



12 At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.



Heart of Agile



<http://alistair.cockburn.us/>

**Change Management is a dolphin,
not a whale**



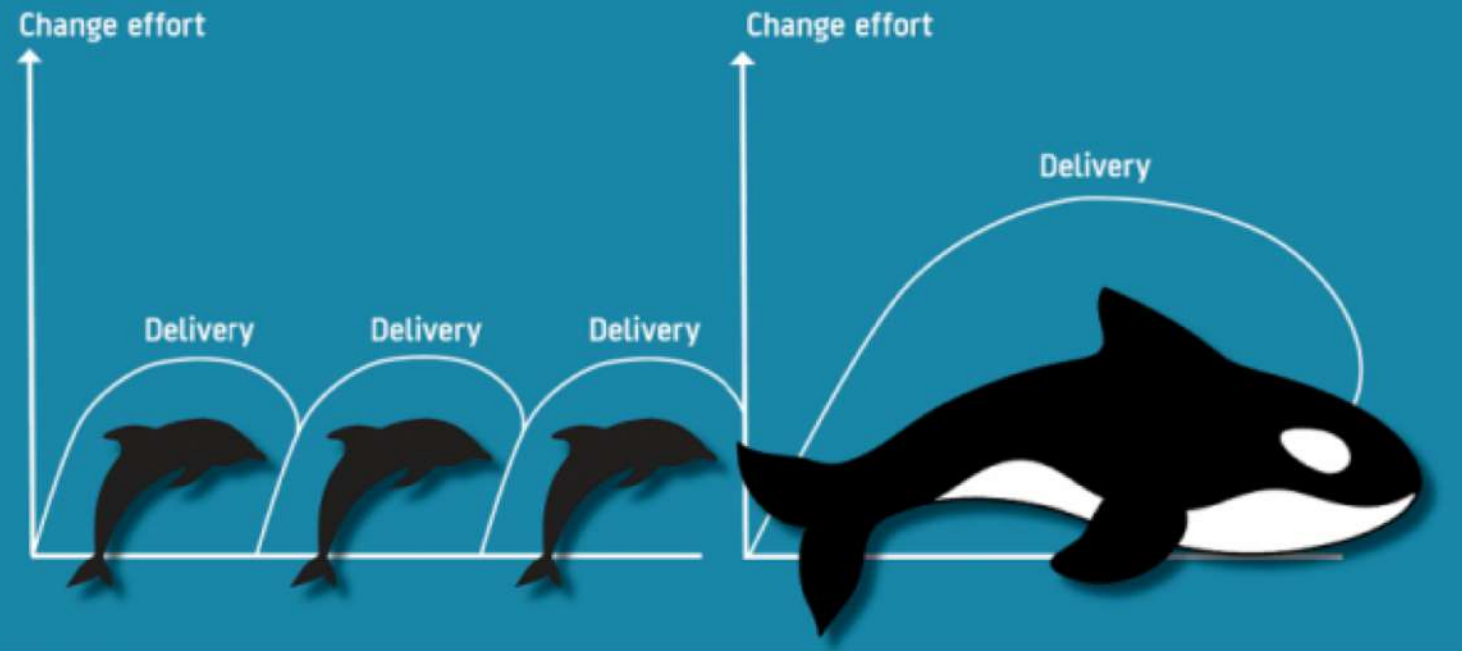


Dolphins surface frequently to take short breaths, communicate and ensure contact with the rest of the pod. The level of effort required for each breath is small. Whales, on the other hand, tend to take long breaths, dive deep and stay submerged for long periods of time. The level of effort to surface for another breath is significant.

Communication is **extremely important** when it comes to making change stick. It takes time for people to hear, understand, and believe the message. And if they don't particularly like what they hear, then it takes even more time for them to come to terms with the concept of change.

Leaders should make it clear why, how and when a change is being instituted, not just to employees but **to anybody else who might be affected by it as well**. It keeps employees informed of progress, reinforces the change benefits, answers the “what’s in it for me?” question and sets expectations.

Change management is a dolphin, not a whale

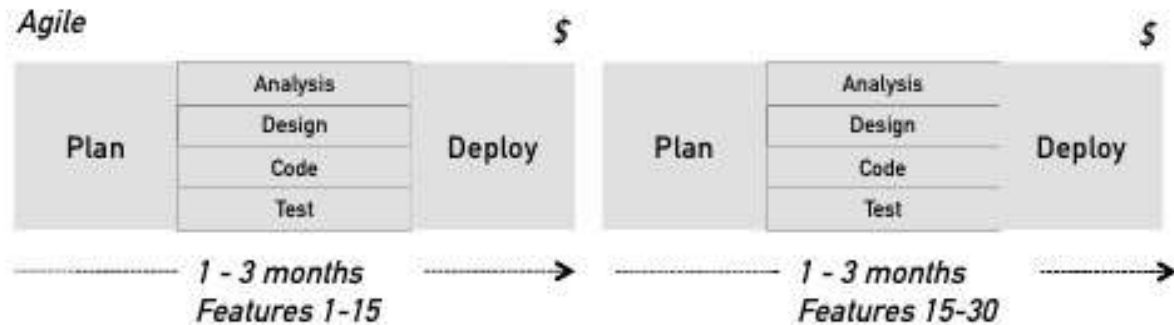


Lifecycles

Waterfall



Agile



Source: The Art of Agile Development by Shore and Warden

DEVELOPMENT: AGILE VS WATERFALL

where the risk lies

WATERFALL



AGILE



Deliverable Product



Chance of failure



Project Run Rate

Agile development uses an iterative approach to reduce risk and create products that are "launchable" throughout development.



SCRUM

A Software Development Framework



How the customer explained it



How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported

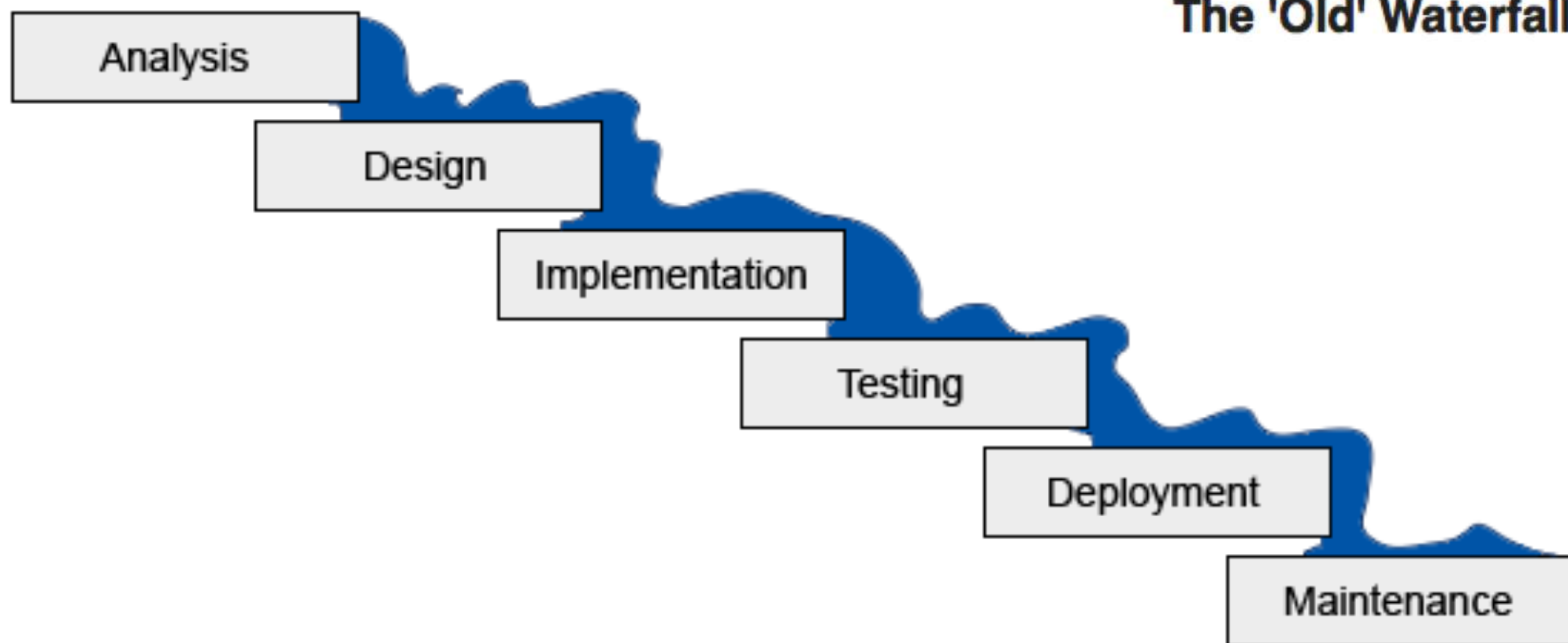


What marketing advertised



What the customer really needed

The 'Old' Waterfall Method



- What if a requirement was missed and only identified during testing?
- What if a deployment issue required changes to the system design?
- What if testing highlighted a misunderstanding with requirements?

The Agile: Scrum Framework at a glance

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Product Owner



The Team



Product Backlog

Team selects starting at top as much as it can commit to deliver by end of Sprint

Sprint Planning Meeting



Sprint Backlog



1-4 Week Sprint

Sprint end date and team deliverable do not change



Scrum Master



Burndown/up Charts



Daily Scrum Meeting

Every 24 Hours



Sprint Review



Finished Work



Sprint Retrospective

Scrum Roles

Product Owner



Scrum Master



Delivery Team



Product Owner : The empowered central point of product leadership. One of the three roles on a Scrum team; the single voice of the stakeholder community to the Scrum team. The product owner defines what to do and what order to do it.



**Responsible for the
product success**

**Cares about user needs
and business goals**

**"Own" the product on
behalf of business owners**

**Good team player -
Collaborates with the
development team and
stakeholders**

**Maintain stakeholders
vision while developing
the product**



Product Owner

**Guide the project in the
correct "direction"**

Development Team : A self-organizing, cross-functional team of people who collectively are responsible for all of the work necessary to produce working, validated assets. One of the three roles that constitute every Scrum team.

- 7-9 persons
- Skill in analysis, development, testing, interface design, database design, architecture, documentation and so on.
- Avoids multi-tasking across multiple products or projects
- Avoids changing team members

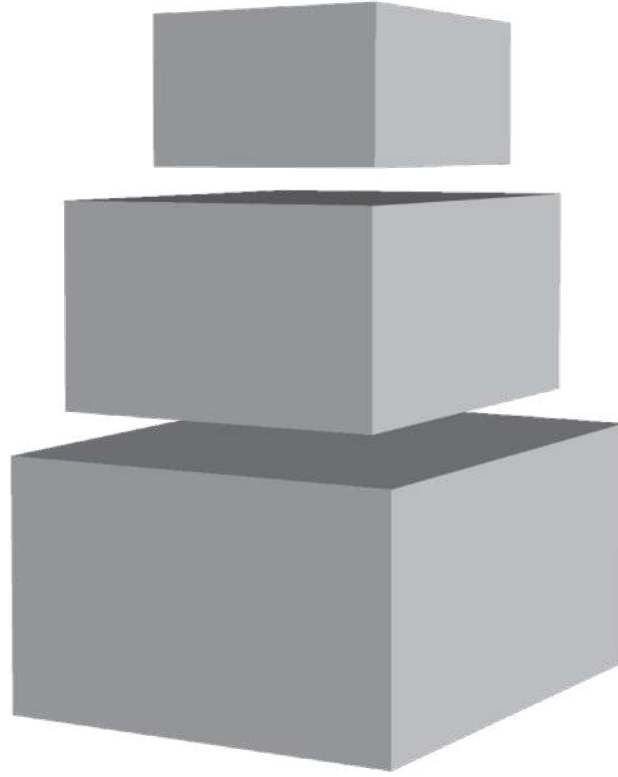


ScrumMaster: The coach, facilitator, impediment remover, and servant leader of the Scrum team. The ScrumMaster is one of the three roles on a Scrum team. The ScrumMaster provides process leadership and helps the Scrum team and the rest of the organization develop their own high-performance, organization-specific Scrum approach.

Scrum Master



- Servant Leader
- Monitoring & Tracking
- Reporting & Communication
- Process Check Master
- Quality Master
- Resolve Impediments
- Resolve Conflicts
- Shield the team
- Performance Feedback



Product Backlog
(Features)

Product Backlog : A prioritized list of requirements with estimated times to turn them into completed product functionality. Estimates are more precise the higher an item is in the Product Backlog priority.

Product Backlog Item : Functional requirements, non-functional requirements, and issues, prioritized in order of importance to the business and dependencies, and estimated. The precision of the estimate depends on the priority and granularity of the Product Backlog item, with the highest priority items that may be selected in the next sprint being very granular and precise..

New Estimates of Effort Remaining at end of Day...									
Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database		5						
	create webpage (UI)		8						
	create webpage (Javascript logic)		13						
	write automated acceptance tests		13						
	update buyer help webpage		3						
	...								
Improve transaction processing performance	merge DCP code and complete layer-level tests		5						
	complete machine order for pRank		8						
	change DCP and reader to use pRank http API		13						

Account lockout after three attempts

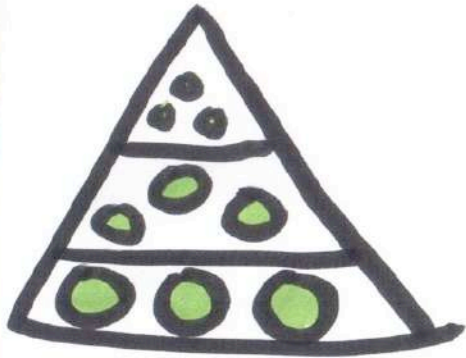
Acceptance Criteria:

Small

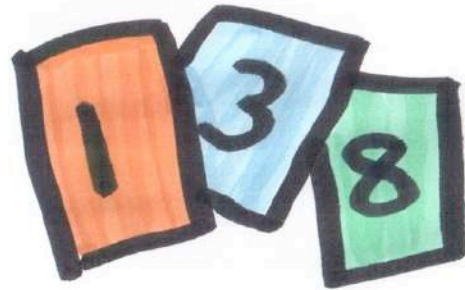
A good Product Backlog is

D. E. E. P

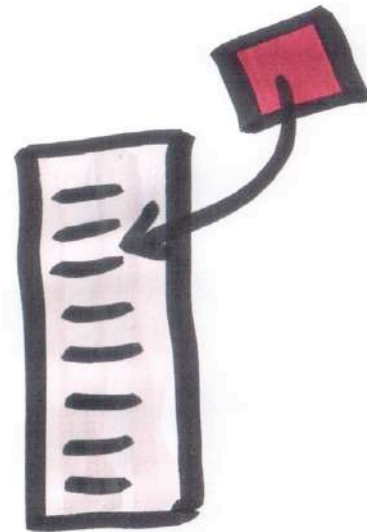
Detailed
appropriately



Estimated



Emergent

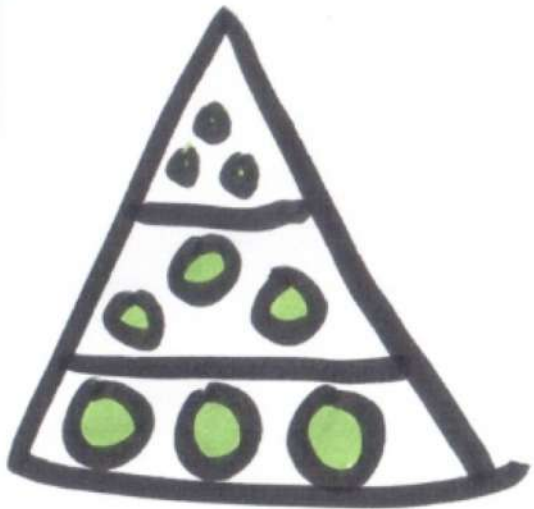


Prioritised



D.

Detailed
appropriately



The top priority items are more fine-grained and detailed than the lower priority items, since the former will be worked on sooner than the latter. For example, the top 10% of the backlog may be composed of very small, well-analyzed items, and the other 90% much less so.

The item for the current release need **to have estimates**, and furthermore, should be considered for **re-estimation** each Sprint as everyones learns and new information arises. **The Team provides the Product Owner with effort estimates for each item** on the Product Backlog, and perhaps also technical risk estimates. The **Product Owner** and other business stakeholders **provide information on the value of the product requests**, which may include revenue gained, costs reduced, business risks, importance to various stakeholders, and more.

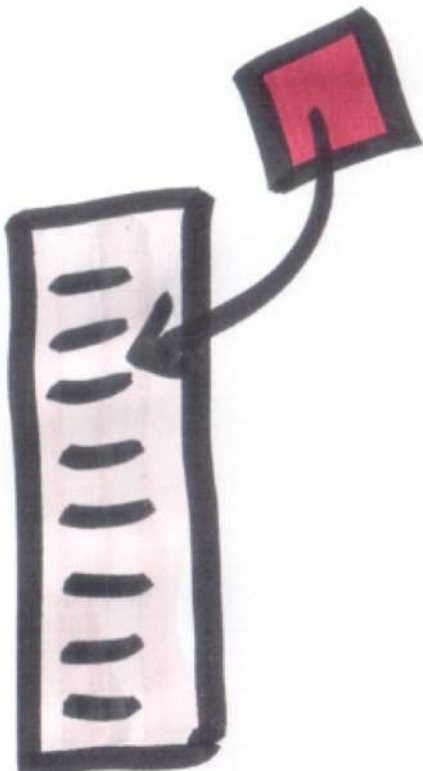
E.

Estimated



E.

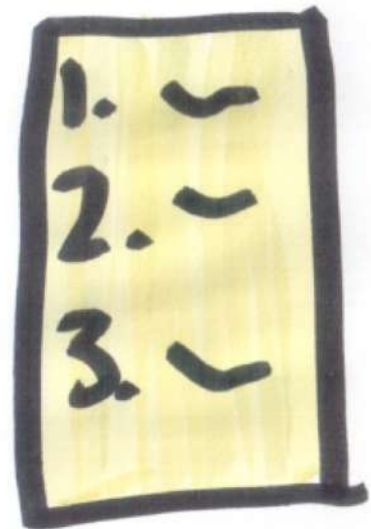
Emergent

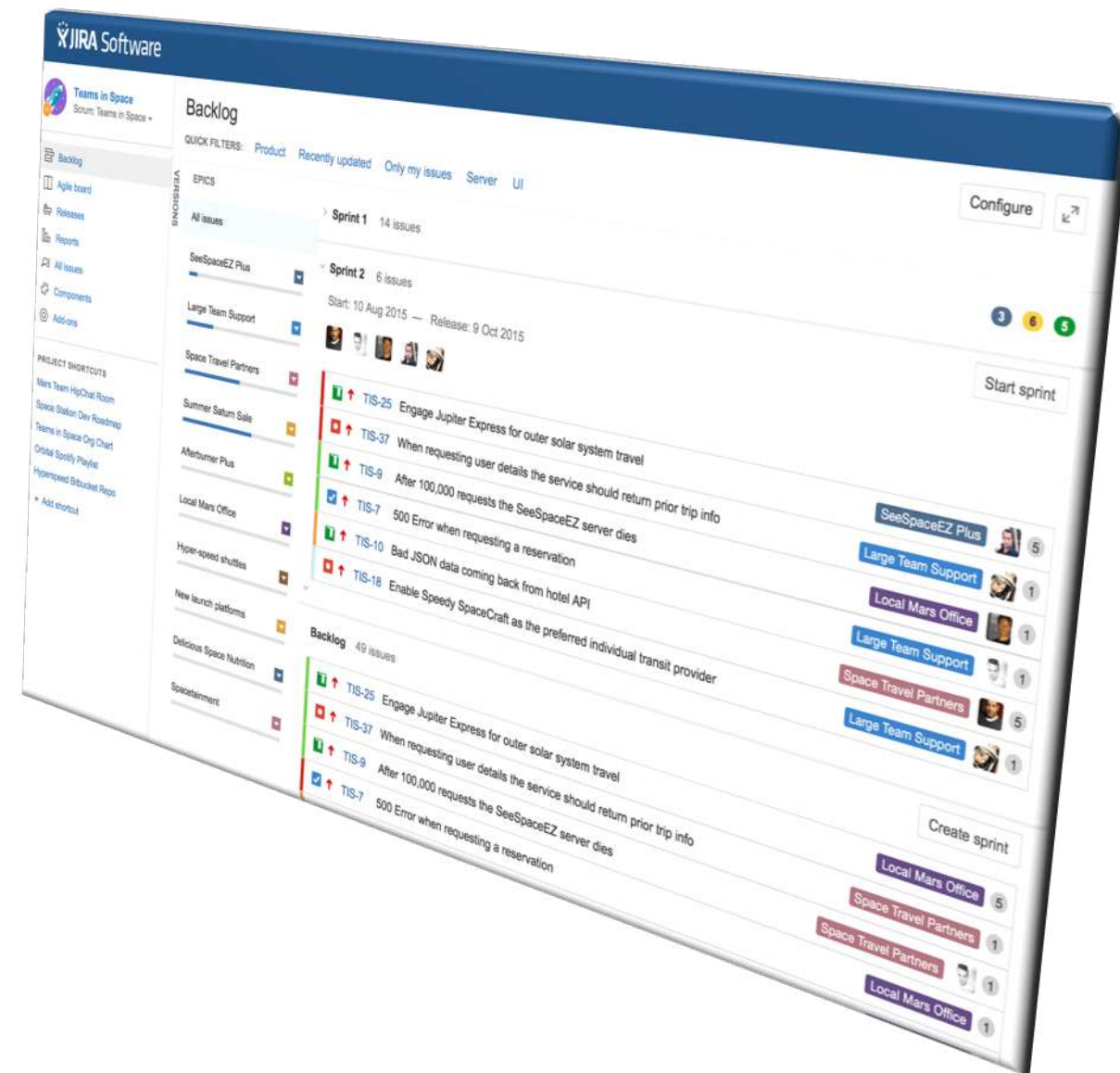


In response to learning and variability, **the Product Backlog is regularly refined**. Each Sprint, items may be added, removed, modified, split, and changed in priority. Thus, **the Product Backlog is continuously updated** by the Product Owner to reflect changes in the needs of the customer, new ideas or insights, moves by the competition, technical hurdles that appear, and so forth.

The items at the top of the Product Backlog are prioritized or ordered in a 1-N order. In general, **the highest-priority items should deliver the most *bang for your buck***: lots of bang (business value) for low buck (cost). Another motivation to increase the priority of an item is to ***tackle high risk early, before the risk attack you.***

P
Prioritised





Definition of Done



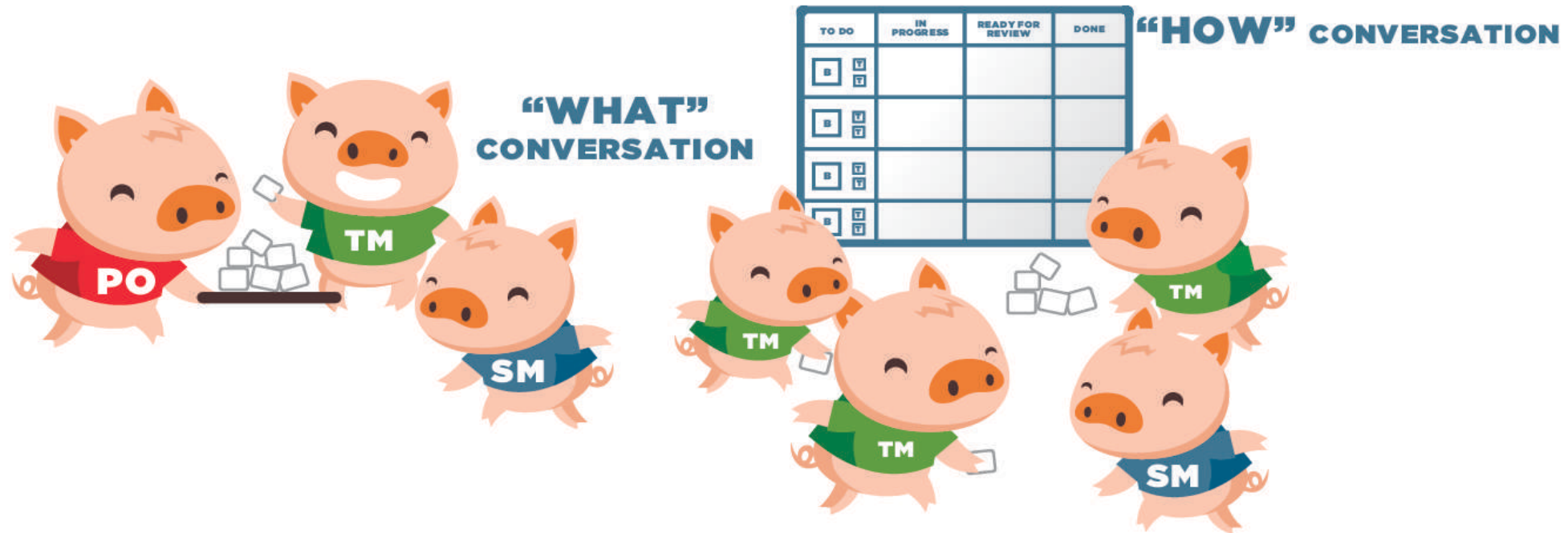
1. A checklist of the types of work that the team is expected to successfully complete by the end of the sprint, before it can declare its work to be potentially shippable.
2. Sometimes described as the acceptance criteria that apply to all product backlog items.

Definition of Done - Assert Quality

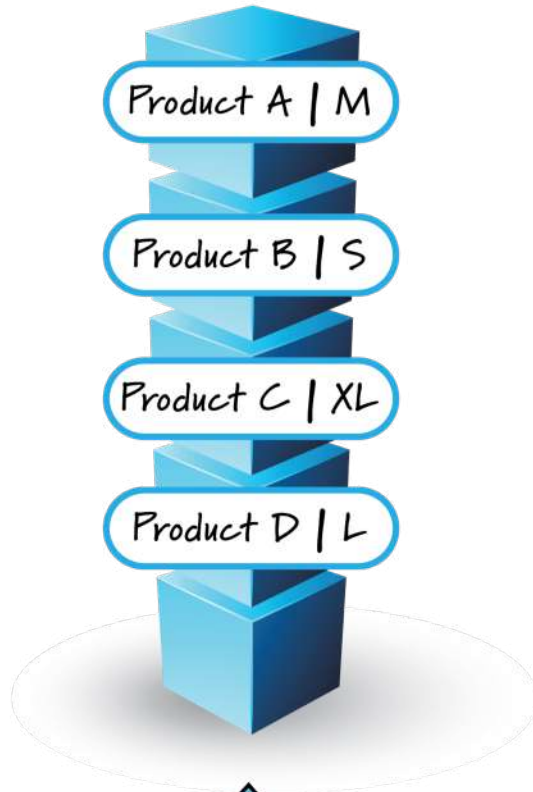
- | | |
|---|--|
| ☐ Acceptance defined criteria for each user story | ☐ Code checked in with reference to US#/Task# |
| ☐ Unit tests written and passed | ☐ Integration test written & passes |
| ☐ Code compiles with no errors and no warnings | ☐ Test code reviewed |
| ☐ New code doesn't break existing code | ☐ Environment requirements documented |
| ☐ Test case review (Dev to review test case written) | ☐ Interface document updated/added and checked in to SVN |
| ☐ Architectural impact assessed and artifacts updated if necessary | ☐ Acceptance criteria verified complete |
| ☐ Comments in code | ☐ All P1-P3 bugs for the story are closed |
| ☐ Error codes added | ☐ Test approves user story |
| ☐ Code reviewed by peer | ☐ Story demonstrated to product owner and accepted on Target Platform |

The Product Owner and Team need to agree on a Definition of Done, which is a subset of the activities that are needed for creating a Potentially Shippable Product Increment. The Team will plan their Sprint work according to this DoD.

Sprint Planning. A time when the Scrum team gathers to agree on a sprint goal and determine what subset of the product backlog it can deliver during the forthcoming sprint. During sprint planning, a sprint backlog is produced to help the team acquire confidence that it can deliver the committed product backlog items.



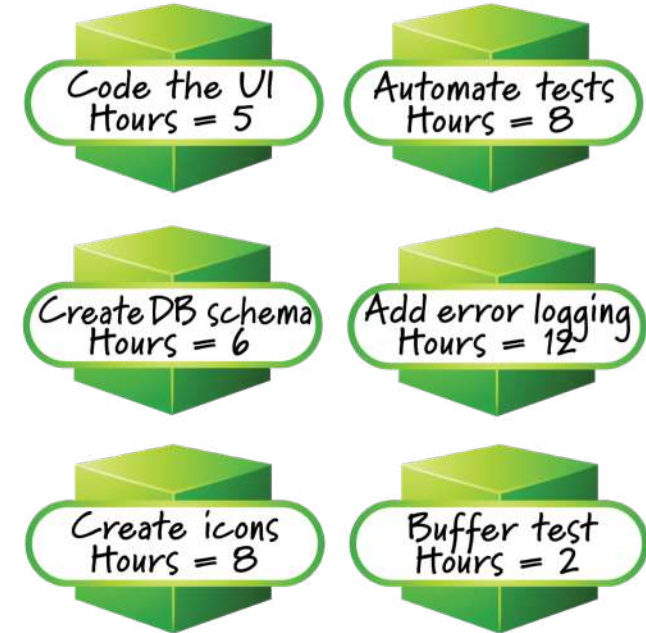
Portfolio backlog



Product backlog



Sprint backlog tasks



Item	Portfolio backlog	Product backlog	Sprint backlog tasks
Unit	T-shirt sizes	Story points / ideal days	Ideal hours / effort-hours
When	Portfolio planning	Product backlog grooming	Sprint planning



Sprint Planning Part One : (PO, Team, SM)

- PO and Team review the high-priority items in the Product Backlog that the PO is interested.
- Focuses on understanding WHAT the PO want and WHY they need.
- PO may leave but they MUST be available during Part Two

Sprint Planning Part Two : (Team, SM, PO is optional)

- Focuses on HOW to implement the items
- Forecasts the amount of items they can complete by the end of sprint

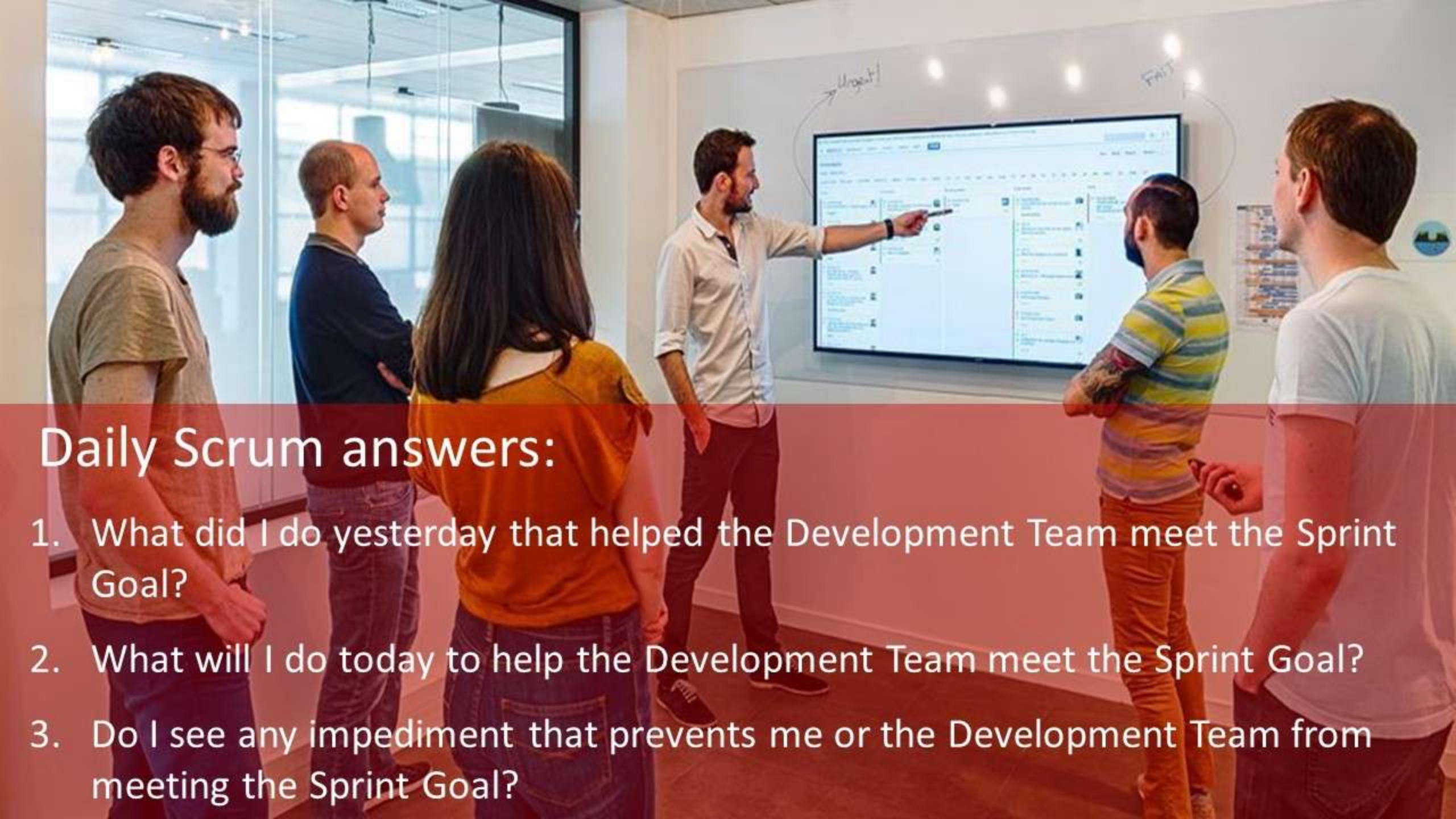
Two hours max. for each part.

New Estimates of Effort Remaining at end of Day...									
Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database		5						
	create webpage (UI)		8						
	create webpage (Javascript logic)		13						
	write automated acceptance tests		13						
	update buyer help webpage		3						
	. . .								
Improve transaction processing performance	merge DCP code and complete layer-level tests		5						
	complete machine order for pRank		8						
	change DCP and reader to use pRank http API		13						



Daily Scrum

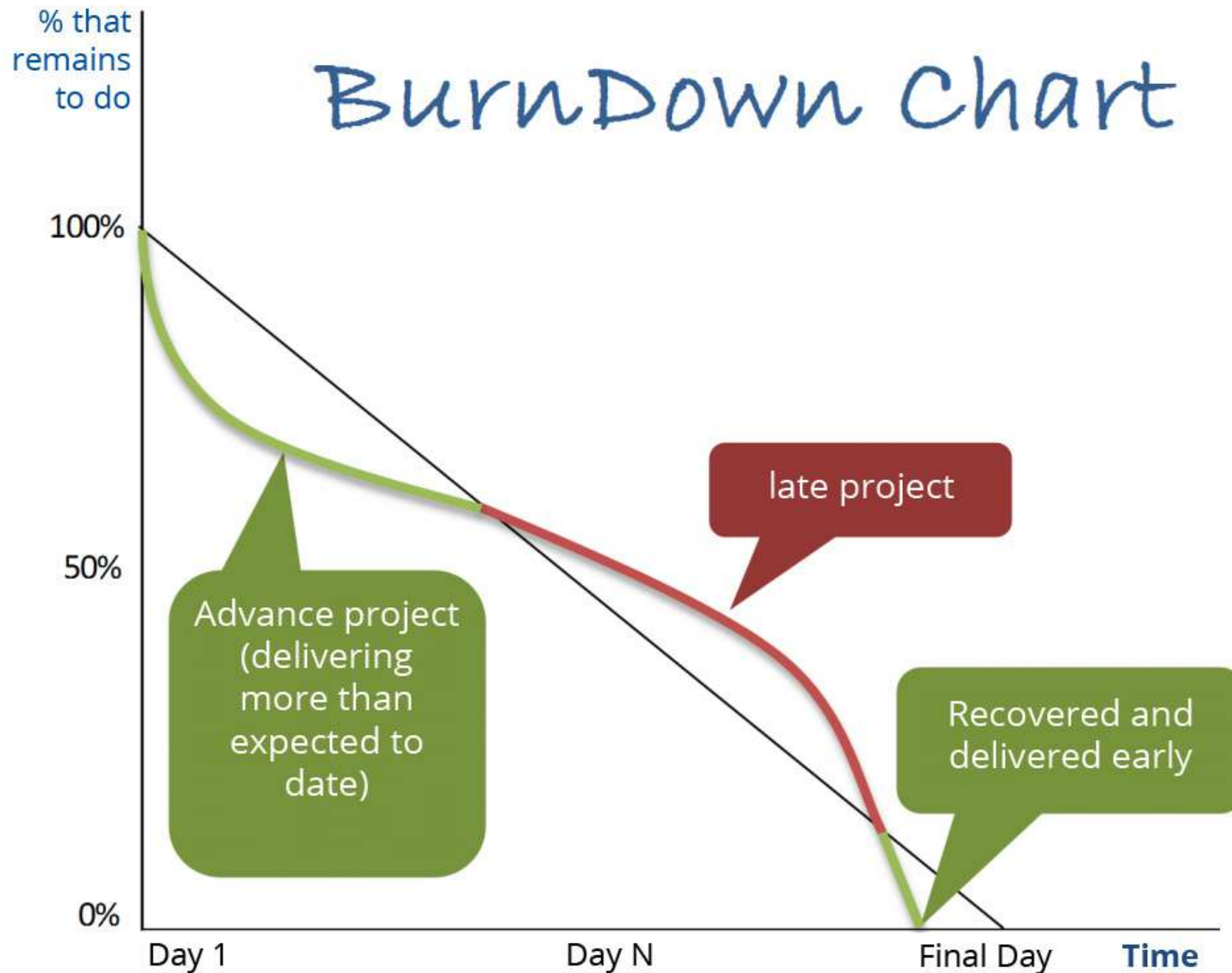
- Yesterday I completed...
- Today, I plan to...
- My impediment is....



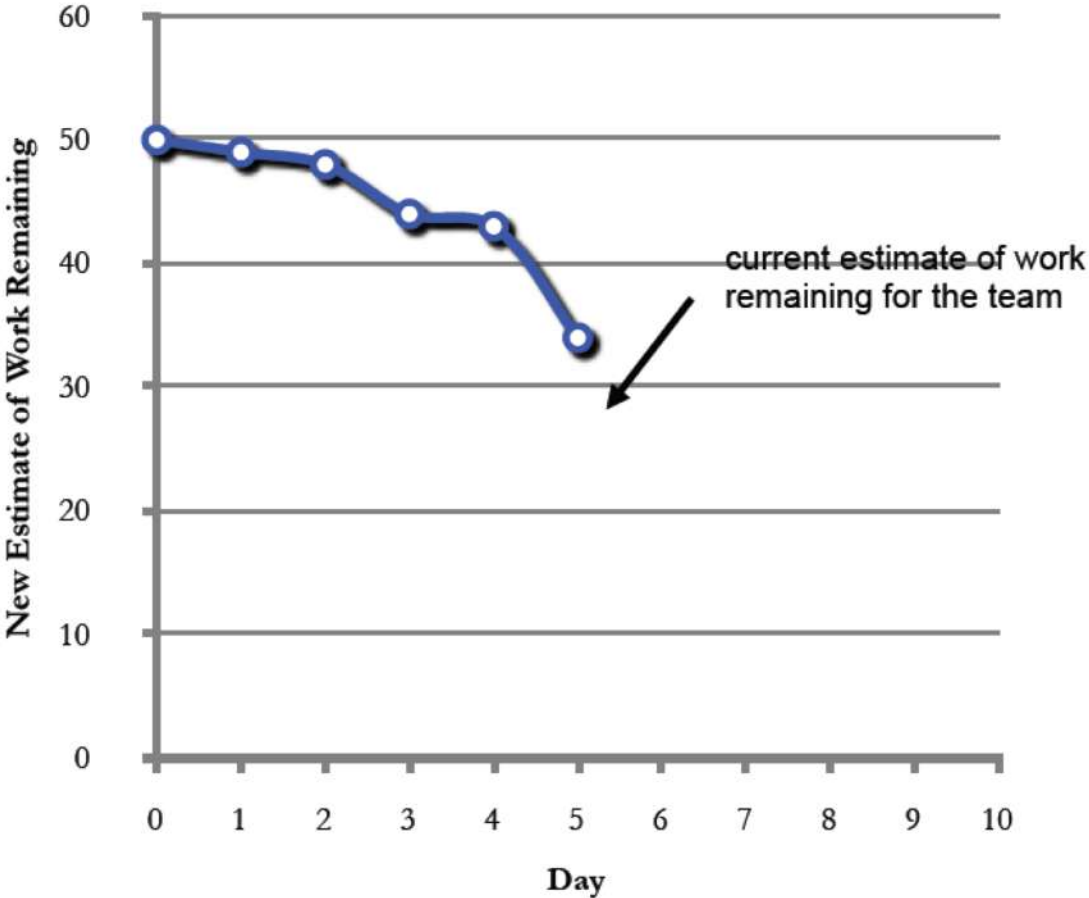
Daily Scrum answers:

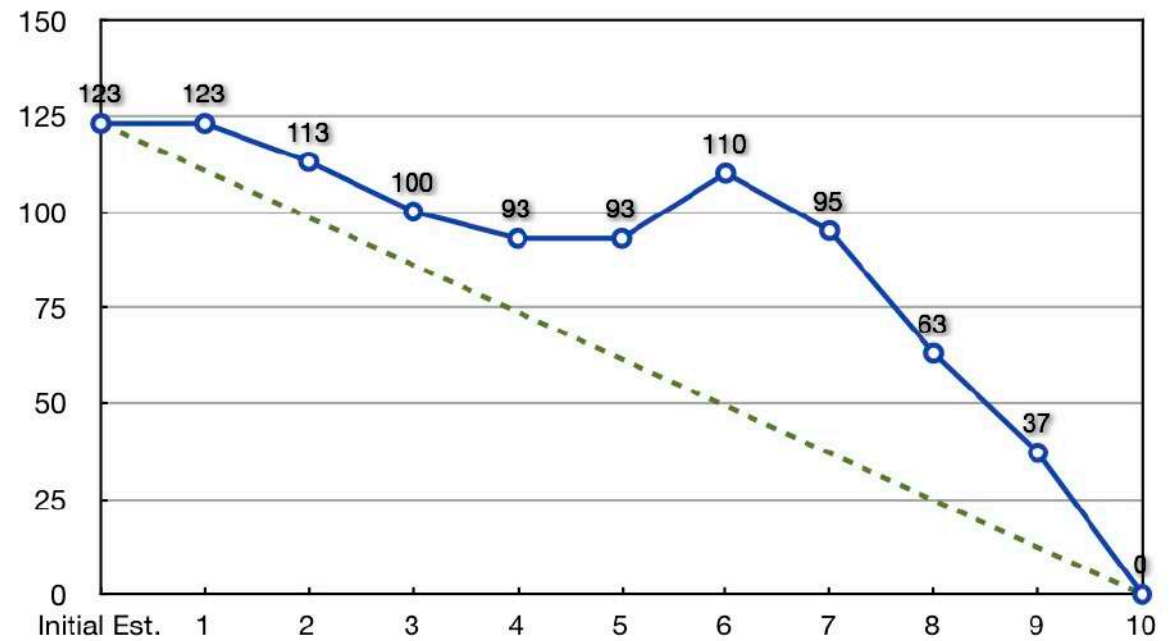
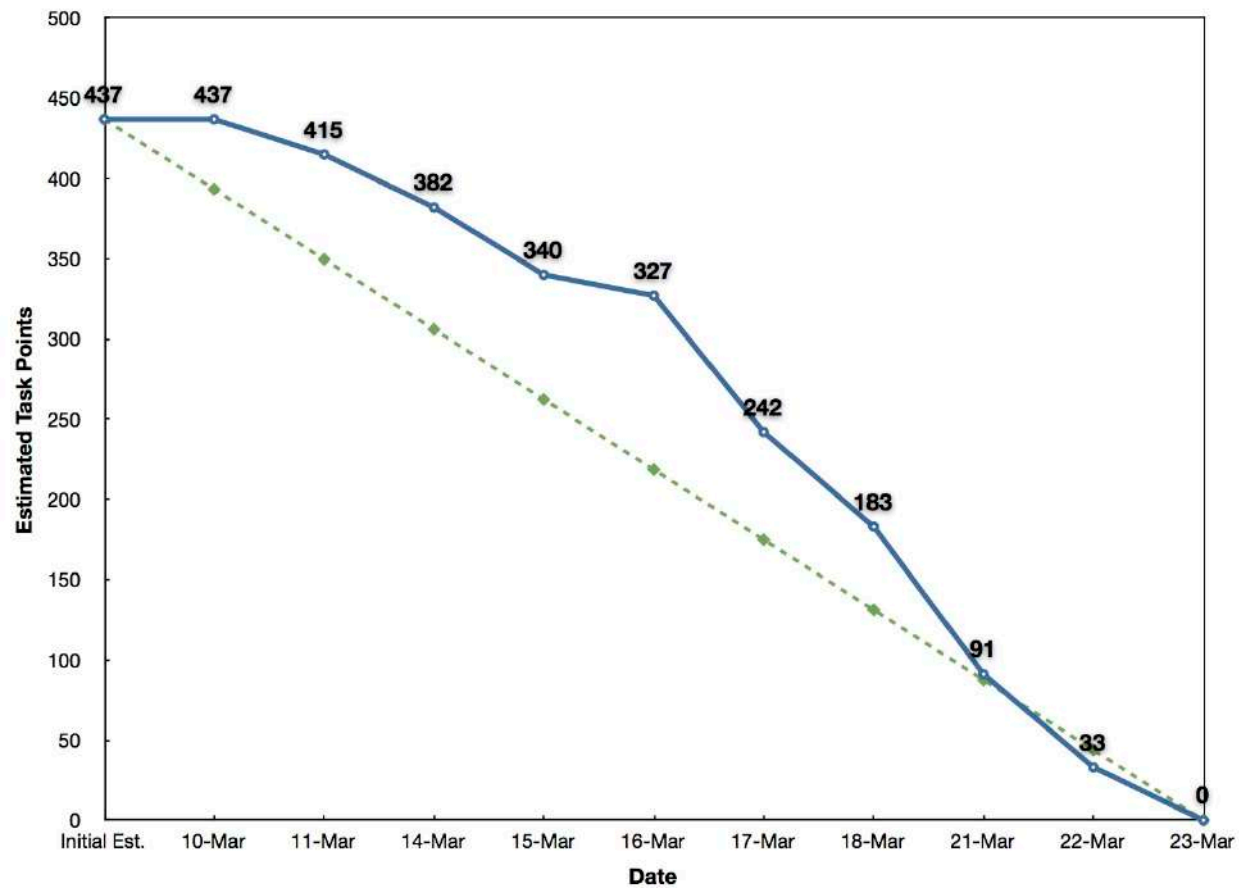
1. What did I do yesterday that helped the Development Team meet the Sprint Goal?
2. What will I do today to help the Development Team meet the Sprint Goal?
3. Do I see any impediment that prevents me or the Development Team from meeting the Sprint Goal?

BurnDown Chart

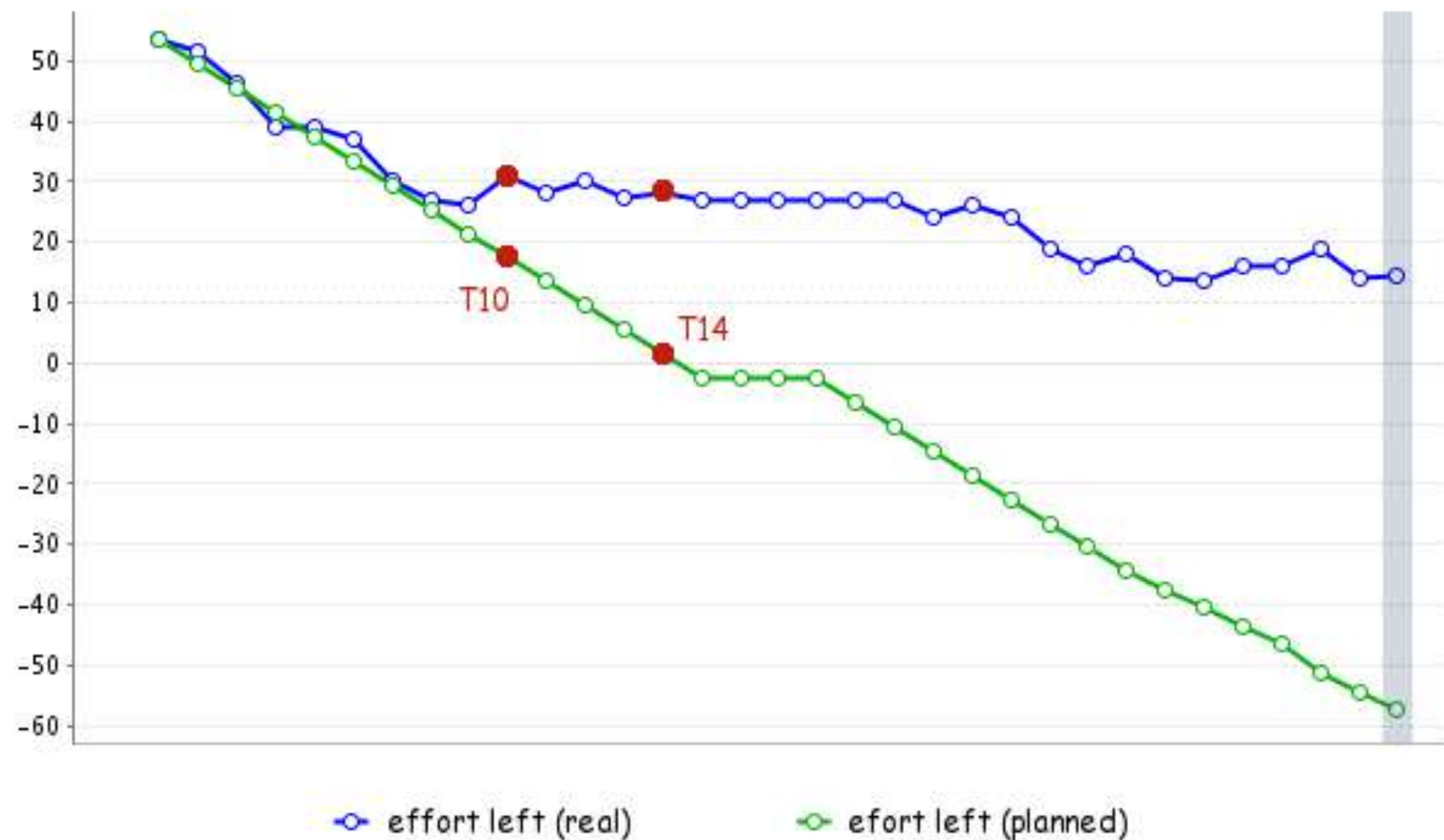


New Estimates of Effort Remaining at end of Day...									
Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database	Sanjay	5	4	3	0	0	0	
	create webpage (UI)	Jing	3	3	3	2	0	0	
	create webpage (Javascript logic)	Tracy & Sam	2	2	2	2	1	0	
	write automated acceptance tests	Sarah	5	5	5	5	5	0	
	update buyer help webpage	Sanjay & Jing	3	3	3	3	3	0	
	ind complete layer-level tests		5	5	5	5	5	5	
	order for pRank		3	3	8	8	8	8	
	reader to use pRank http API		5	5	5	5	5	5	
		...	...						
		Total	50	49	48	44	43	34	

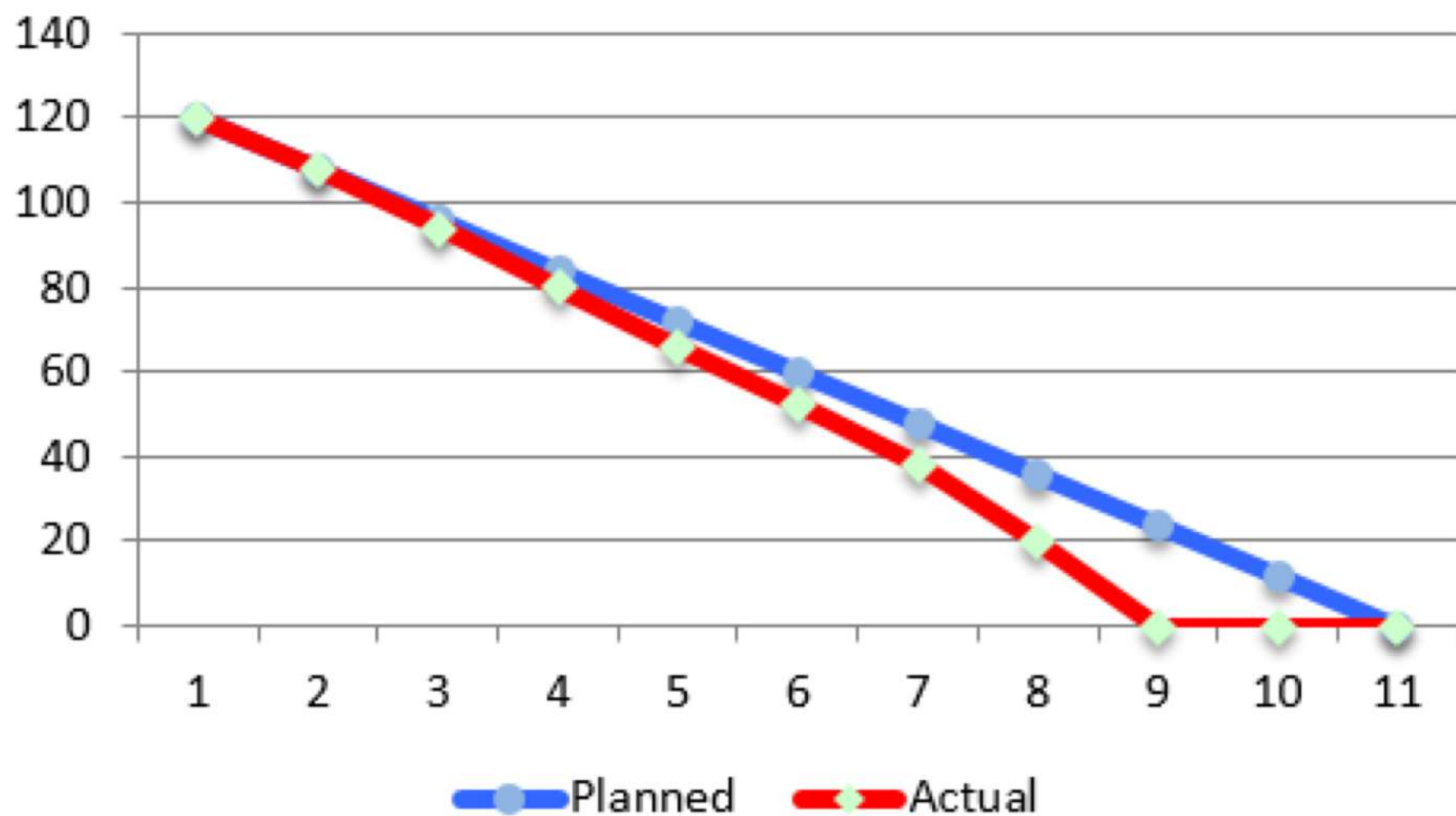




Burndown chart - initial effort as a base (type I)



Sprint 2 Burndown



105414

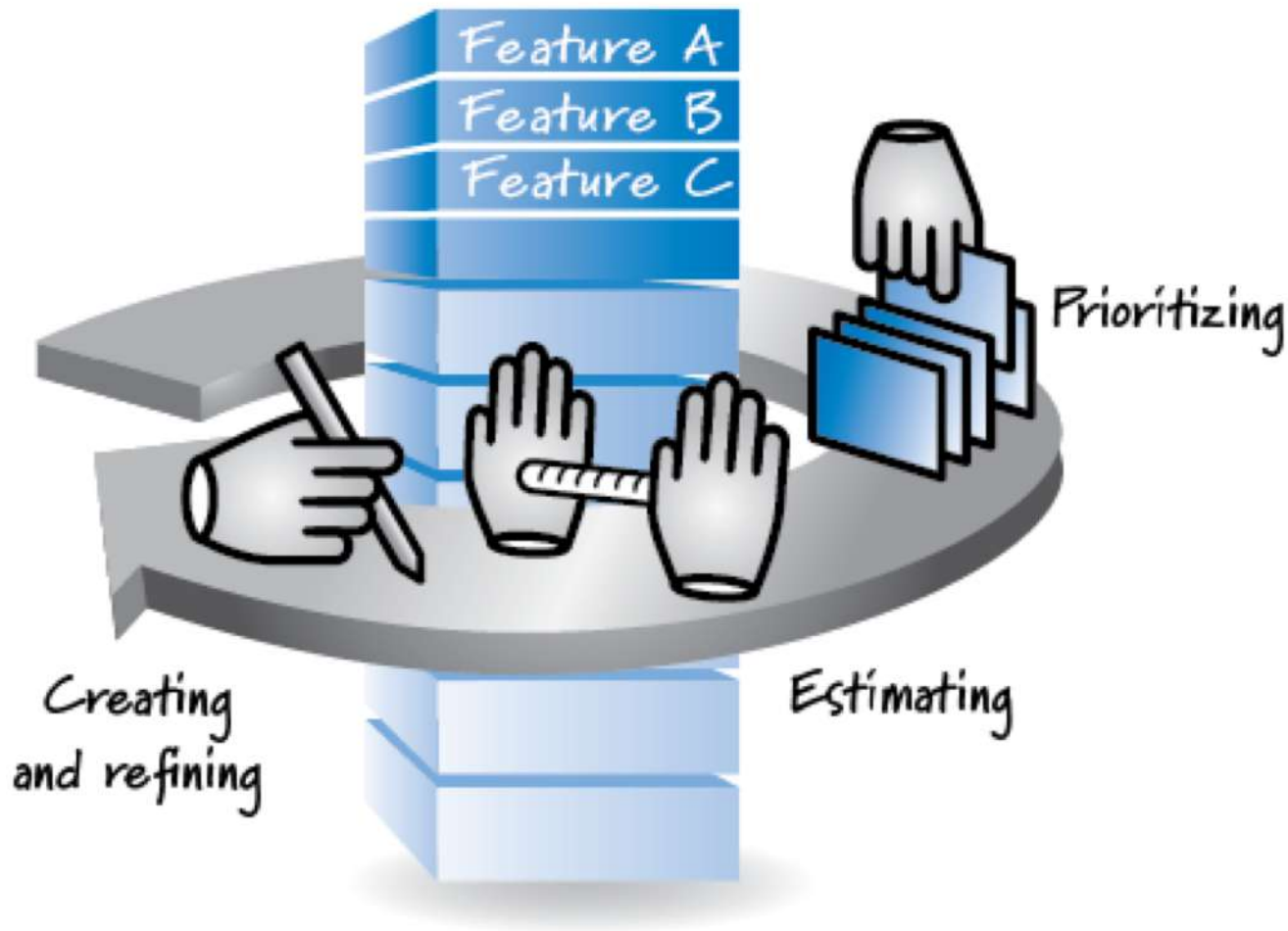
IMPEDIMENTS

(continued)

6. Island	production = 900
(exclusive) (Japan)	700

Chlorine

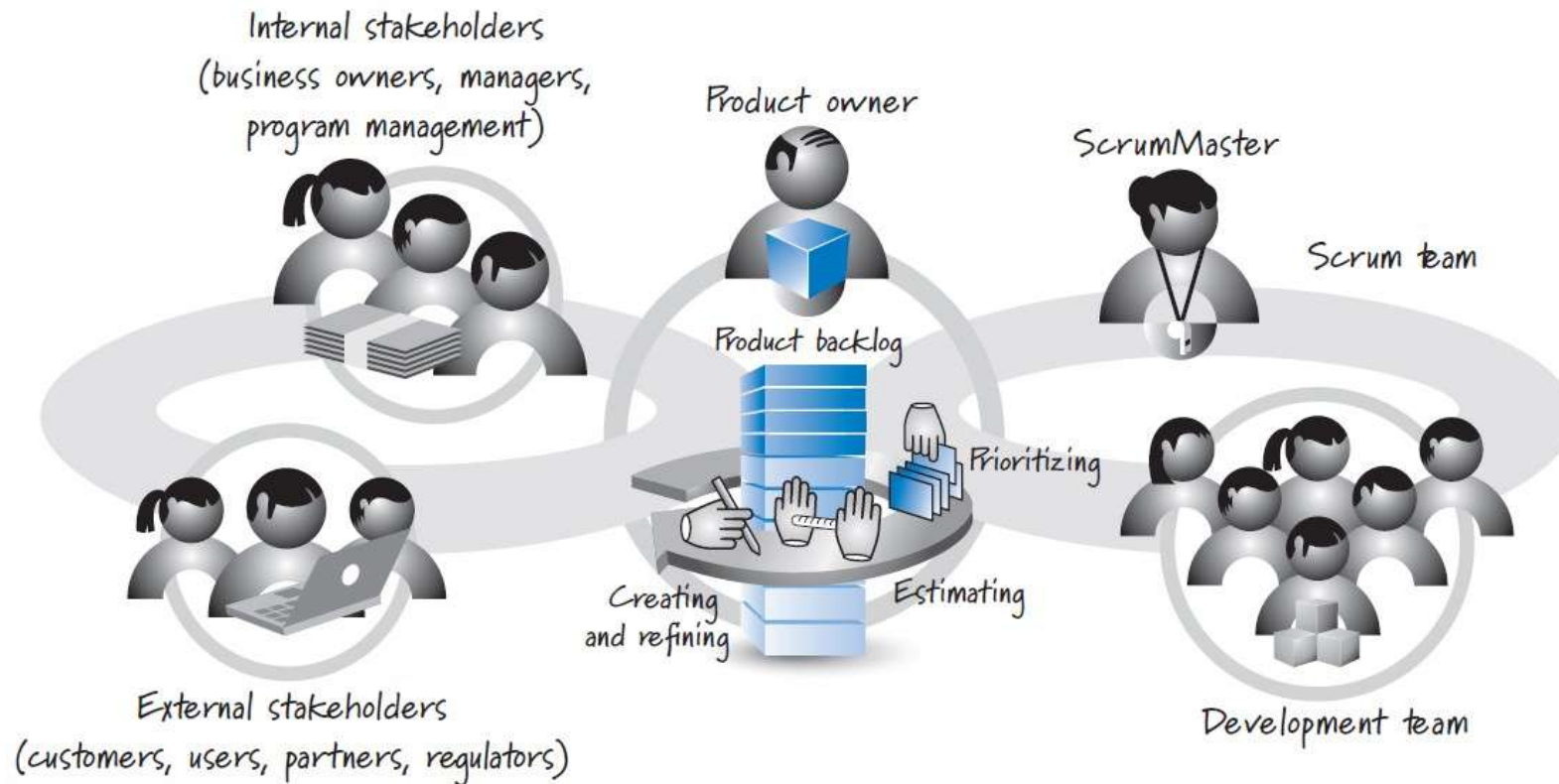
Product Backlog refinement



- Split big items, analyze items, re-estimate and re-prioritize for future Sprints.
- Team, PO, SM (may not attend)
- Perhaps one day is spent on refinement.

Sprint Review





- One hour per week of Sprint
- Anyone present is free to ask question and give input.
- Is not DEMO, but inspect and adapt

SPRINT RETROSPECTIVE



Retrospective

Sprint Retrospective – It's about the TEAM!

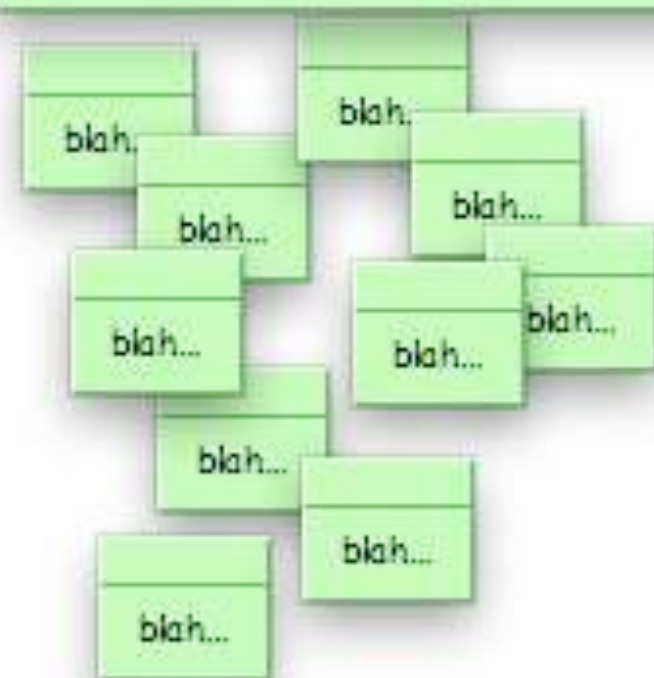
- Typically after each sprint
- Typically 30 minutes
- Whole team participates
 - ScrumMaster
 - Product Owner
 - Team
- 3 Questions:
 - What did we do well?
 - What could we do better?
 - What are the action items for improvement going forward?



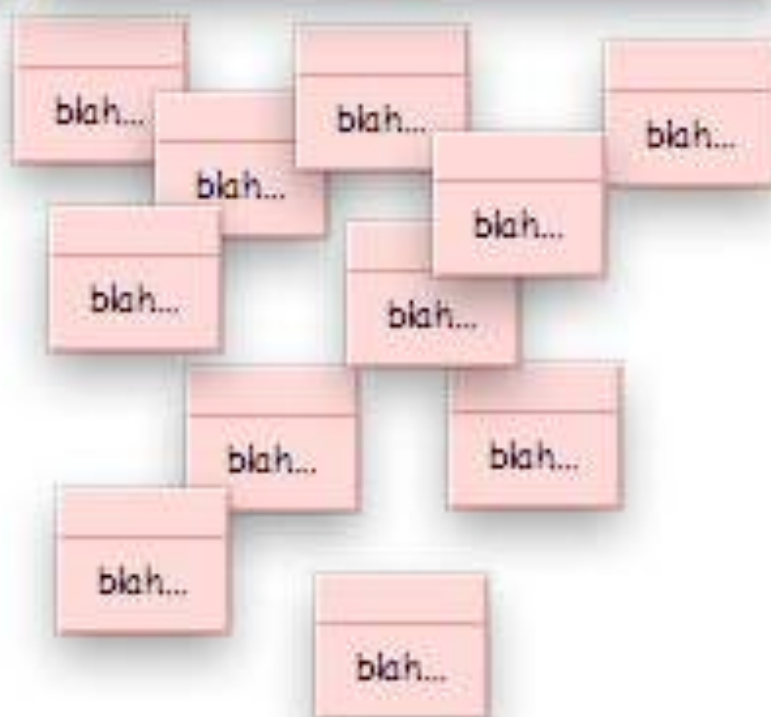
Start / Stop / Continue

- Whole team participates
 - What would we like to start doing?
 - What would we like to stop doing?
 - What would we like to continue doing?

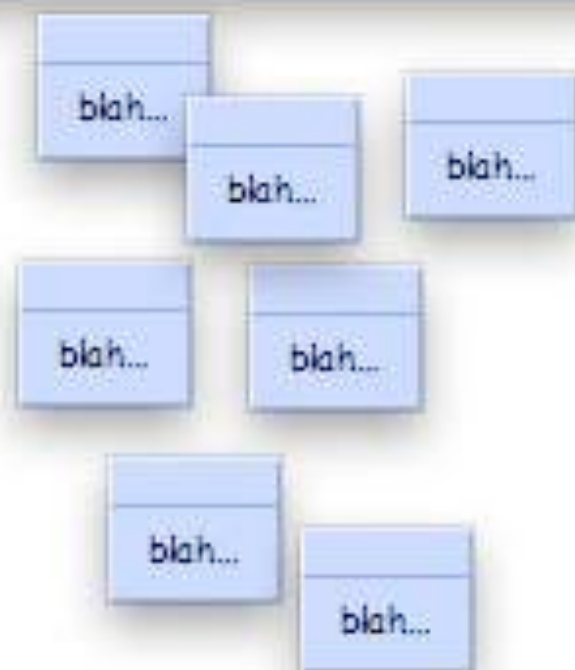
What Went Well



What Could Have Been Improved



Actions



Starting the next Sprint

The Agile: Scrum Framework at a glance

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Product Owner



The Team



Product Backlog

Team selects starting at top as much as it can commit to deliver by end of Sprint

Sprint Planning Meeting



Sprint Backlog



1-4 Week Sprint

Sprint end date and team deliverable do not change

Scrum Master

Burndown/up Charts



Every 24 Hours



Daily Scrum Meeting



Sprint Review



Finished Work



Sprint Retrospective